LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [05] – [Relasi antar Kelas]

I Kadek Rio Adi pranata Kusuma

103132400029

S1-SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Diagram kelas adalah diagram yang digunakan untuk memodelkan kelas-kelas dalam sistem beserta hubungan antar kelas tersebut. Simbol kotak kelas terdiri atas baris pertama untuk nama kelas, baris kedua untuk atribut, dan baris berikutnya untuk operasi atau *method*. Selain itu, diagram ini menggunakan *access modifier* seperti tanda + untuk *public*, - untuk *private*, dan # untuk *protected*. Dalam Pemrograman Berorientasi Objek, kelas-kelas dapat memiliki berbagai jenis hubungan, yaitu asosiasi, agregasi, komposisi, serta generalisasi dan spesialisasi yang berkaitan dengan pewarisan.

Hubungan asosiasi merupakan relasi antara dua kelas terpisah yang terbentuk melalui objek dan komunikasi antar keduanya, seperti hubungan antara kelas Dosen dan Mahasiswa. Agregasi adalah hubungan bagian dan keseluruhan (*part-of*) di mana kedua kelas tetap dapat diciptakan secara terpisah, contohnya kelas Jurusan dan Mahasiswa. Komposisi merupakan bentuk khusus agregasi yang lebih kuat, di mana kelas bagian tidak akan bisa ada tanpa adanya kelas utama, seperti hubungan Komputer dengan CPU, Monitor, dan Mouse. Terakhir, generalisasi adalah hubungan di mana suatu kelas merupakan turunan dari kelas lainnya.

B. Guided

1. [Asosiasi - Dosen]

```
public class Dosen {
   private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    //Getter
    public String getKodeDosen() {

        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai) {
    s.setNilai(nilai);
    }
```

```
    public int getScore(Mahasiswa s) {
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

[Asosiasi - Mahasiswa]

```
public class Mahasiswa {
   private String nim;
    private String nama;
    private int nilai;

    //Setter
    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public void setNilai(int nilai) {
        this.nilai = nilai;
    }

    //Getter
    public int getNilai(){
        return this.nilai;
    }

    public String getNama() {
        return this.nama;
    }

    public String getNim() {
```

```
        return this.nim;
    }
}
```

Output :

```
------------------------< com.Rio:ProjectModul05 >------------------------
Building ProjectModul05 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul05 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul

--- compiler:3.15.0:compile (default-compile) @ ProjectModul05 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
Nim :130118383
Nama :Budi
Nilai :90
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.476 s
Finished at: 2026-10-18T13:48:40+07:00
------------------------------------------------------------------------
```

2. [Agregasi - Jurusan]

```
public class Jurusan {
    //artinya milik itu sendiri
    private static int jumlahMahasiswa;
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    //constructor
    public Jurusan (String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    //Behaviour
    public void addMahasiswa(Mahasiswa m){
        this.daftarMahasiswa[jumlahMahasiswa]= m;
        jumlahMahasiswa++;
    }

    //menampilkan
    public void displayMahasiswa(){
```

```
        System.out.println("Kode Jurusan: " + this.kodeJurusan);
        System.out.println("Nama Jurusan: " + this.namaJurusan);

        System.out.println("Daftar Mahasiswa: ");

        for (int i = 0; i<jumlahMahasiswa;i++){
           System.out.println(i+1 + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    //Getter
    public String getKodeJurusan(){
        return kodeJurusan;
    }

  public String getNamaJurusan(){
      return namaJurusan;
  }
    //Setter

  public void setKodeJurusan(String kodeJurusan){
      this.kodeJurusan = kodeJurusan;
  }

  public void setNamaJurusan(String namaJurusan){
      this.namaJurusan = namaJurusan;
  }
}
```

[Agregasi - Mahasiswa]

```
public class Mahasiswa {
   private String NIM, Nama;

    //constructor
    public Mahasiswa (String NIM, String Nama){
        this.NIM = NIM;
        this.Nama = Nama;
    }

    //Getter
    public String getNIM(){
        return NIM;
    }

    public String getNama(){
        return Nama;
    }
```

```
    //Setter
    public void setNIM(String NIM){
        this.NIM = NIM;
    }

    public void setNama(String Nama){
        this.Nama = Nama;
    }
}
```

3. [Komposisi - Komputer]

```
public class Komputer {
    private Processor processor;
    private Monitor monitor;
    private Mouse mouse;

    public Komputer(String brandCpu, int kecepatanCpu, String brandMonitor, String
brandMouse, String tipeMouse){
        this.processor = new Processor(brandCpu, kecepatanCpu);
        this.monitor = new Monitor(brandMonitor);
        this.mouse = new Mouse(brandMouse, tipeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== SPESIFIKASI KOMPUTER ===");
        this.processor.displaySpec();
        this.monitor.DisplaySpec();
        this.mouse.displaySpec();
    }
}
```

[Komposisi - Monitor]

```
public class Monitor {
    private String brand;

    public Monitor(String brand){
        this.brand = brand;
    }

    public void DisplaySpec() {
        System.out.println(this.brand);
    }
}
```

[Komposisi – Mouse]

```
public class Mouse {
    private String brand, tipe;
    public Mouse(String brand, String tipe){
        this.brand = brand;
        this.tipe = tipe;
    }

    public void displaySpec(){
        System.out.println(this.brand + ", " + this.tipe);
    }
}
```

[Komposisi - Processor]

```
public class Processor {
    private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output :

```
-----------------------< com.Rio:ProjectModul05 >-----------------------
Building ProjectModul05 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul05 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-reg-01\modul-05-relasi

--- compiler:3.15.0:compile (default-compile) @ ProjectModul05 ---
Recompiling the module because of changed source code.
Compiling 14 source files with javac [debug release 26] to target\classes

--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
Kode Jurusan: SD
Nama Jurusan: Sains Data
Daftar Mahasiswa:
1. 103112400148 Rendhi
2. 103112400149 John
=== SPESIFIKASI KOMPUTER ===
Intel Core i5, 3200
LG 24 Inch
Logitech, Wireless
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  2.883 s
Finished at: 2026-10-18T14:20:04+07:00
------------------------------------------------------------------------
```

4. [Main]

```
package com.rio.projectmodul05;
import com.rio.projectmodul05.Guided2.komposisi.Komputer;
import com.rio.projectmodul05.Guided2.agregasi.*;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }


    public static void runGuided1(){}

    public static void runGuided2(){
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD","Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();

    }

    public static void runGuided3(){
```

```
        Komputer pc = new Komputer("Intel Core i5",3200,"LG 24
Inch","Logitech","Wireless");
        pc.displaySpecKomputer();
    }
}
```

C. Unguided

1. Soal Unguided 1

Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

Relasi Antar Kelas: Komposisi, Agregasi, dan Asosiasi

Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan `ArrayList` atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (`ItemPesanan`) pasti memiliki rincian harga (`RincianHarga`) yang mencatat harga dasar dan pajak (10%).
- Objek `RincianHarga` harus di-instansiasi langsung di dalam constructor `ItemPesanan`. Objek `RincianHarga` tidak pernah dibuat di `main`. Jika `ItemPesanan` dihapus, maka `RincianHarga` miliknya ikut hilang.

Kebutuhan Tugas 1

Buatlah dua class: `RincianHarga` dan `ItemPesanan`.

Field yang disarankan:

Class	Field	Jenis	Tipe	Keterangan
`RincianHarga`	`hargaDasar`	Instance field	`double`	Harga awal sebelum pajak
`RincianHarga`	`pajak`	Instance field	`double`	Nilai nominal pajak (10% dari harga dasar)
`ItemPesanan`	`namaItem`	Instance field	`String`	Nama menu (misal: "Iced Americano")
`ItemPesanan`	`rincian`	Instance field	`RincianHarga`	Objek internal rincian harga (**Komposisi**)

Method yang disarankan:

Class	Method	Return	Keterangan
`RincianHarga`	`getHargaTotal()`	`double`	Mengembalikan nilai `hargaDasar + pajak`
`ItemPesanan`	`getNamaItem()`	`String`	Mengembalikan nama item pesanan

`ItemPesanan`	`getRincian()`	`RincianHarga`	Mengembalikan objek rincian harga

Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah `Pesanan` menampung beberapa objek `ItemPesanan` menggunakan array standar.
- Objek `ItemPesanan` dibuat terpisah terlebih dahulu (di `main`), lalu dimasukkan satu per satu ke dalam `Pesanan` melalui method `tambahItem()`. Jika `Pesanan` dibatalkan atau dihapus, objek `ItemPesanan` tersebut secara konsep tetap ada.

Kebutuhan Tugas 2

Buatlah class `Pesanan`.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`nomorMeja`	Instance field	`int`	Nomor meja pelanggan
`daftarItem`	Instance field	`ItemPesanan[]`	Array penampung item pesanan (**Agregasi**)
`jumlahItem`	Instance field	`int`	Penanda jumlah item yang sudah masuk ke array

Method yang disarankan:

Method	Return	Keterangan
`tambahItem(ItemPesanan item)`	`void`	Memasukkan objek `ItemPesanan` dari luar ke dalam array `daftarItem`
`tampilkanPesanan()`	`void`	Menampilkan nomor meja dan seluruh item beserta harga total per item

`getNomorMeja()`	`int`	Mengembalikan nomor meja
`getDaftarItem()`	`ItemPesanan[]`	Mengembalikan array daftar item
`getJumlahItem()`	`int`	Mengembalikan jumlah item yang tersimpan

Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang `Kasir` memproses pembayaran dari sebuah `Pesanan`.
- Class `Kasir` tidak menyimpan objek `Pesanan` sebagai field, melainkan menerima objek `Pesanan` sebagai parameter pada method `prosesPembayaran()`.

Kebutuhan Tugas 3

Buatlah class `Kasir`.

Field yang disarankan:

Field	Jenis	Tipe	Keterangan
`namaKasir`	Instance field	`String`	Nama petugas kasir

Method yang disarankan:

Method	Return	Keterangan
`prosesPembayaran(Pesanan pesanan)`	`void`	Mengakses data `Pesanan`, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**)

Tugas 4: Eksekusi dan Pengujian di `Main`

Buatlah class `Main` untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

Alur program di `main`

1. Buat 1 objek `Kasir` (misal: `"Budi"`).
2. Buat 3 objek `ItemPesanan` secara independen:
   - `"Iced Americano"`, harga dasar: `18000`
   - `"Beef Croissant"`, harga dasar: `25000`
   - `"Matcha Latte"`, harga dasar: `22000`
3. Buat 1 objek `Pesanan` untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek `ItemPesanan` ke dalam objek `Pesanan` menggunakan method `tambahItem()`.
5. Tampilkan rincian pesanan dengan memanggil `tampilkanPesanan()` pada objek `Pesanan`.
6. Jalankan pemrosesan pembayaran dengan memanggil method `prosesPembayaran(pesanan)` dari objek `Kasir`.

Output yang Diharapkan

```
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

Yang Perlu Diperhatikan Saat Mengerjakan

- **Komposisi:** Objek `RincianHarga` wajib di-instansiasi di dalam constructor `ItemPesanan` menggunakan kata kunci `new`. Dilarang melakukan `new RincianHarga(...)` di dalam `main`.

- **Agregasi:** Objek `ItemPesanan` wajib di-instansiasi di `main` terlebih dahulu, kemudian dikirimkan sebagai argumen ke method `tambahItem()`.
- **Asosiasi:** Class `Kasir` dilarang memiliki field bertipe `Pesanan`. Interaksi hanya terjadi saat method `prosesPembayaran(Pesanan pesanan)` dipanggil.

Pesanan - Agregasi

```
package com.rio.projectmodul05.Unguided.Agregasi;
import com.rio.projectmodul05.Unguided.Komposisi.ItemPesanan;
import com.rio.projectmodul05.Unguided.Komposisi.RincianHarga;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem; // Agregasi
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Pesanan penuh, item " + item.getNamaItem() + " tidak
dapat ditambahkan.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);
        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga r = item.getRincian();
            System.out.println((i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + r.getHargaTotal()
                    + " (Harga: " + r.getHargaDasar()
                    + ", Pajak: " + r.getPajak() + ")");
        }
        System.out.println();
    }
```

```
    public int getNomorMeja() {
        return this.nomorMeja;
    }

    public ItemPesanan[] getDaftarItem() {
        return this.daftarItem;
    }

    public int getJumlahItem() {
        return this.jumlahItem;
    }
}
```

Kasir - Asosiasi

```
package com.rio.projectmodul05.Unguided.Asosiasi;
import com.rio.projectmodul05.Unguided.Agregasi.Pesanan;
import com.rio.projectmodul05.Unguided.Komposisi.ItemPesanan;
import com.rio.projectmodul05.Unguided.Komposisi.RincianHarga;

public class Kasir {
    private String namaKasir;

    public Kasir(String namaKasir) {
        this.namaKasir = namaKasir;
    }

    // Asosiasi: Pesanan hanya diterima sebagai parameter, bukan disimpan sebagai
field
    public void prosesPembayaran(Pesanan pesanan) {
        double totalSubtotal = 0;
        double totalPajak = 0;
        ItemPesanan[] items = pesanan.getDaftarItem();

        for (int i = 0; i < pesanan.getJumlahItem(); i++) {
            RincianHarga r = items[i].getRincian();
            totalSubtotal += r.getHargaDasar();
            totalPajak += r.getPajak();
        }
        double totalBayar = totalSubtotal + totalPajak;

        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + pesanan.getJumlahItem());
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
```

```
            System.out.println("==============================");
        }
}
```

ItemPemesanan - Komposisi

```
public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian; // Komposisi

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        // Komposisi: RincianHarga dibuat di dalam constructor ItemPesanan
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

RincianHarga - Komposisi

```
public class RincianHarga {
    private double hargaDasar;
    private double pajak;

    public RincianHarga(double hargaDasar) {
        this.hargaDasar = hargaDasar;
        this.pajak = hargaDasar * 10 / 100; // pajak 10% dari harga dasar
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

Main

```
package com.rio.projectmodul05.Unguided;

import com.rio.projectmodul05.Unguided.Agregasi.Pesanan;
import com.rio.projectmodul05.Unguided.Asosiasi.Kasir;
import com.rio.projectmodul05.Unguided.Komposisi.ItemPesanan;

public class Main {
    public static void main(String[] args) {
        // 1. Kasir
        Kasir kasir = new Kasir("Budi");

        // 2. ItemPesanan dibuat independen di main
        //    (RincianHarga dibuat otomatis di dalam ItemPesanan)
        ItemPesanan item1 = new ItemPesanan("Iced Americano", 18000);
        ItemPesanan item2 = new ItemPesanan("Beef Croissant", 25000);
        ItemPesanan item3 = new ItemPesanan("Matcha Latte", 22000);

        // 3. Pesanan untuk Meja 4, kapasitas 5 item
        Pesanan pesanan = new Pesanan(4, 5);

        // 4. Agregasi: memasukkan item yang sudah ada ke dalam pesanan
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // 5. Tampilkan pesanan
        pesanan.tampilkanPesanan();

        // 6. Asosiasi: kasir memproses pembayaran
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
-----------------------< com.Rio:ProjectModul05 >-----------------------
Building ProjectModul05 1.0-SNAPSHOT
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.4.0:resources (default-resources) @ ProjectModul05 ---
skip non existing resourceDirectory C:\Semester 5\pbo file\PBO\templat repo\praktikum-pbo-ps1ds-05-r

--- compiler:3.15.0:compile (default-compile) @ ProjectModul05 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul05 ---
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
Total time:  1.676 s
Finished at: 2026-10-18T18:09:37+07:00
------------------------------------------------------------------------
```

Penjelasan :

Bagian Unguided ini membangun sebuah aplikasi manajemen pesanan kafe berbasis konsol yang terdiri dari beberapa kelas utama yang saling berelasi:

1. **Kelas RincianHarga (Komposisi) :** Kelas ini bertanggung jawab untuk mengelola harga dasar suatu menu serta menghitung besaran nominal pajak sebesar 10% dari harga dasar tersebut melalui *constructor* dan *method* getHargaTotal().

2. **Kelas ItemPesanan (Komposisi) :** Kelas ini merepresentasikan item menu individu yang dipesan (seperti "Iced Americano", "Beef Croissant", atau "Matcha Latte"). Di dalamnya, kelas ini mendeklarasikan objek RincianHarga secara langsung di dalam *constructor*-nya untuk menerapkan prinsip komposisi.

3. **Kelas Pesanan (Agregasi) :** Kelas ini berfungsi sebagai wadah penampung kumpulan objek ItemPesanan untuk nomor meja tertentu. Menggunakan *array* berkapasitas tetap (ItemPesanan[]), kelas ini menyediakan *method* tambahItem(ItemPesanan item) guna memasukkan item-item pesanan yang sudah dibuat di luar serta *method* tampilkanPesanan() untuk mencetak rincian seluruh daftar pesanan beserta kalkulasi harga per itemnya.

4. **Kelas Kasir (Asosiasi):** Kelas ini merepresentasikan petugas kasir (bernama "Budi") yang bertugas memproses transaksi pembayaran. Melalui *method* prosesPembayaran(Pesanan pesanan), kasir menerima parameter objek Pesanan, mengambil daftar item di dalamnya, mengakumulasikan total harga dasar dan total pajak, lalu mencetak struk pembayaran akhir secara terperinci.

5. **Kelas Main (Pengujian):** Kelas utama ini menjadi pusat perangkaian seluruh program. Di sinilah objek Kasir diinisialisasi, beberapa objek ItemPesanan dibuat secara independen dengan harga masukan masing-masing, objek Pesanan untuk Meja 4 disiapkan, item dimasukkan melalui agregasi, hingga akhirnya perintah untuk menampilkan daftar pesanan dan pemrosesan pembayaran oleh kasir dieksekusi secara berurutan hingga menghasilkan *output* sistem kafe yang sukses.

D. Kesimpulan

Berdasarkan seluruh rangkaian praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman mengenai relasi antar kelas (*class relationship*) sangat krusial dalam Pemrograman Berorientasi Objek (OOP) untuk membangun struktur kode yang terorganisir, modular, dan mencerminkan skenario dunia nyata secara akurat. Melalui implementasi sistem manajemen pesanan kafe, terlihat jelas perbedaan fungsional serta karakteristik dari masing-masing bentuk relasi, mulai dari hubungan komposisi yang menunjukkan ikatan kuat dengan siklus hidup objek bagian yang bergantung penuh pada kelas utamanya, hubungan agregasi yang merepresentasikan wadah kumpulan objek mandiri secara lebih longgar, hingga hubungan asosiasi yang mencerminkan interaksi fungsional antarobjek independen melalui pengiriman parameter *method* tanpa kepemilikan atribut permanen.